```
import json, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from openai import OpenAI
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score

client = OpenAI(api_key = "sk-proj-hDJsiqMQMh93vHQe_YizsTuU15PNyToZ0GcY1fVRVfNij2f_g57Nh-Bv4b-GqHPAj-lV4u9dsqT3BlbkFJY9J8b3HzYbEfMyx
```

LLM reads and understanding the optimization task

```
def get_hyperparameter_constraints(X, y):
        n_samples, n_features = X.shape
        n_classes = len(np.unique(y))

        return {
                "n_estimators": {"type": "int", "range": [10, 500]},    # larger is rarely harmful, just slower
                "max_depth": {"type": "int", "range": [2, int(np.log2(n_samples))]},    # log2(n) is a natural ceiling
                "min_samples_split": {"type": "int", "range": [2, max(20, n_samples // 50)]},    # scales with dataset size
                "max_features": {"type": "float", "range": [0.1, 1.0]},    # always 0-1
        }
```

```
from sklearn.datasets import load_iris, load_wine, load_breast_cancer

DATASET_REGISTRY = {
        "iris": load_iris,
        "wine": load_wine,
        "breast_cancer": load_breast_cancer,
}

DATASET_NAME = "wine"
dataset = DATASET_REGISTRY[DATASET_NAME]()
X_data, y_data = dataset.data, dataset.target


task_context = {
        "model": "RandomForestClassifier",
        "task": "classification",
        "dataset": DATASET_NAME,
        "metric": "accuracy",
        "lower is better": False,
        "hyperparameter_constraints": get_hyperparameter_constraints(X_data, y_data)
}
```

```
# iris = load_iris()
# X_data, y_data = iris.data, iris.target

def blackbox_evaluation(config: dict, X_data, y_data) -> float:
    clf = RandomForestClassifier(
            n_estimators = int(config["n_estimators"]),
            max_depth = int(config["max_depth"]),
            min_samples_split =   int(config["min_samples_split"]),
            max_features = float(config["max_features"]),
            random_state = 42
    )
    score = cross_val_score(clf, X_data, y_data, cv=5, scoring="accuracy").mean() # the data is split and evaluated 5 ti
    return round(float(score), 4) # 0.9667
```

convert the observation history into a table the LLM can read.

This is what replaces gpr.fit(X_sample, Y_sample).

```
def build_context(observed_configs: list[dict], observed_scores: list[float]) -> str:
    # observed_configs: history of parameter trials
    context = (
            f"Task: Tune{task_context['model']} on {task_context['dataset']} dataset.\n"
            f"Metric: {task_context['metric']} (higher is better).\n\n"
            f"Hyperparameter constraints:\n"
    )
    for hp, info in task_context["hyperparameter_constraints"].items():
        context += f"{hp}: {info['type']} in [{info['range'][0]}, {info['range'][1]}]\n"
```

```
    if observed_configs:
        context += f"\n Past {len(observed_configs)} evaluatoins (sorted by score):\n"
        paired = sorted(zip(observed_scores, observed_configs), key = lambda x: x[0], reverse=True) # LLM immediately sees
        for score, config in paired:
            config_str = ",".join(f"{k}={v}" for k, v in config.items())
            context += f"score={score:.4f} | {config_str}\n"
    return context
```

propose candidates

```
def llm_propose_candidates(observed_configs, observed_scores, n_candidates=5) -> list[dict]:
    context = build_context(observed_configs, observed_scores) # every iteration observed are updated, so to remake contex
    prompt = f"""{context}
        Based on the observations above, propose {n_candidates} DIVERS hyperparameter configurations to evaluate next.
        Mix exploration (try untested regions) and exploitation (refine near best configs).
        Respond ONLY with a valid JSON object in this format:
        {{"candidates": [
            {{"n_estimators": 100, "max_depth": 5, "min_samples_split": 4, "max_features": 0.5}},
            ...
        ]}}
    """
    response = client.chat.completions.create(
            model = "gpt-4o",
            max_tokens = 200,
            messages=[{"role": "user", "content": prompt}],
            response_format = {"type": "json_object"}
    )
    raw = json.loads(response.choices[0].message.content)
    # raw_content = response.choices[0].message.content
    # print(f"[DEBUG] raw response:", raw_content)

    candidates = raw["candidates"]
    clipped = []
    constraints = task_context["hyperparameter_constraints"]
    for c in candidates:
            clipped.append({
                    hp: (
                            round(float(np.clip(c[hp],
                                    constraints[hp]["range"][0],
                                    constraints[hp]["range"][1]
                            )), 2)
                            if constraints[hp]["type"] == "float"
                            else int(np.clip(c[hp],
                                    constraints[hp]["range"][0],
                                    constraints[hp]["range"][1]
                            ))
                    )
                    for hp in constraints
            })
    return clipped
```

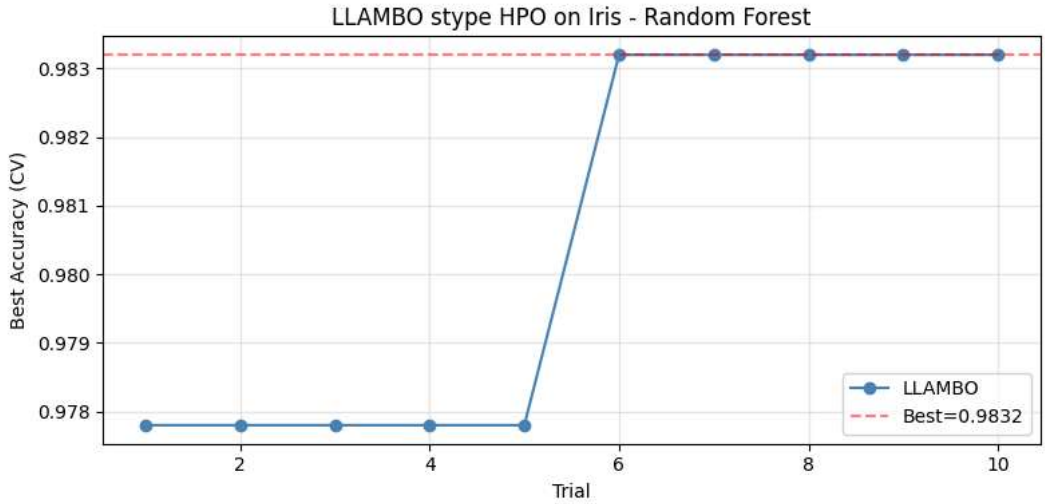
```
def llm_score_and_select(candidates: list[dict], observed_configs, observed_scores) -> dict:
    context = build_context(observed_configs, observed_scores)
    candidates_str = json.dumps(candidates, indent=2)
    prompt = f"""{context}
    You are a surrogate model. Score each candidate configuration below by its predicted {task_context['metric']}.
    Candidates:
    {candidates_str}
    Respond ONLY with a JSON object in this format:
    {{"scores": [score1, score2, ...]}}"""

    response = client.chat.completions.create(
            model = "gpt-4o",
            max_tokens = 200,
            messages=[{"role": "user", "content": prompt}],
            response_format = {"type": "json_object"}
    )
    raw = json.loads(response.choices[0].message.content)
    scores = raw["scores"]
    if isinstance(scores, dict):
        scores = list(scores.values())[0]

    best_idx = int(np.argmax(scores))
    print(f"LLM scores: {[f'{s:.2f}' for s in scores]}")
    print(f" -> selected candidates #{best_idx}: {candidates[best_idx]}")
    return candidates[best_idx]
```

initialization: random configuration to warm start

```
def random_init(n=5) -> list[dict]:
    configs = []
    for _ in range(n):
        configs.append({
                "n_estimators": int(np.random.randint(10, 300)),
                "max_depth": int(np.random.randint(2, 20)),
                "min_samples_split": int(np.random.randint(2, 20)),
                "max_features": round(float(np.random.uniform(0.1, 1.0)), 2),
        })
    return configs
```

```
n_itial = 5
n_trials = 10
n_candidates = 5
observed_configs = random_init(n_itial)
observed_scores = [blackbox_evaluation(c, X_data, y_data) for c in observed_configs]
best_score = max(observed_scores)

history = []
for trial in range(n_trials):
    print(f"\n [Trial{trial+1}/{n_trials}]")
    candidates = llm_propose_candidates(observed_configs, observed_scores, n_candidates)
    best_candidate = llm_score_and_select(candidates, observed_configs, observed_scores)
    score = blackbox_evaluation(best_candidate, X_data, y_data)
    print(f"Blackbox evaluation -> score: {score:.4f}")
    observed_configs.append(best_candidate)
    observed_scores.append(score)
    if score > best_score:
        best_score = score
        print(f"New best:{best_score:.4f}")

    history.append(best_score)
    print(f"Best so far:{best_score:.4f}")
```

```
 [Trial1/10]
LLM scores: ['0.98', '0.98', '0.97', '0.93', '0.94']
 -> selected candidates #1: {'n_estimators': 50, 'max_depth': 7, 'min_samples_split': 10, 'max_features': 0.25}
Blackbox evaluation -> score: 0.9722
Best so far:0.9778

 [Trial2/10]
LLM scores: ['0.97', '0.97', '0.97', '0.95', '0.96']
 -> selected candidates #2: {'n_estimators': 200, 'max_depth': 7, 'min_samples_split': 2, 'max_features': 0.5}
Blackbox evaluation -> score: 0.9721
Best so far:0.9778

 [Trial3/10]
LLM scores: ['0.98', '0.97', '0.97', '0.96', '0.96']
 -> selected candidates #0: {'n_estimators': 31, 'max_depth': 7, 'min_samples_split': 11, 'max_features': 0.19}
Blackbox evaluation -> score: 0.9778
Best so far:0.9778

 [Trial4/10]
LLM scores: ['0.98', '0.97', '0.93', '0.97', '0.97']
 -> selected candidates #0: {'n_estimators': 31, 'max_depth': 7, 'min_samples_split': 11, 'max_features': 0.2}
Blackbox evaluation -> score: 0.9778
Best so far:0.9778

 [Trial5/10]
LLM scores: ['0.98', '0.97', '0.97', '0.97', '0.97']
 -> selected candidates #0: {'n_estimators': 31, 'max_depth': 7, 'min_samples_split': 11, 'max_features': 0.18}
Blackbox evaluation -> score: 0.9778
Best so far:0.9778

 [Trial6/10]
LLM scores: ['0.98', '0.97', '0.97', '0.98', '0.97']
 -> selected candidates #0: {'n_estimators': 35, 'max_depth': 7, 'min_samples_split': 11, 'max_features': 0.19}
Blackbox evaluation -> score: 0.9832
New best:0.9832
Best so far:0.9832

 [Trial7/10]
LLM scores: ['0.98', '0.98', '0.97', '0.97', '0.97']
 -> selected candidates #0: {'n_estimators': 35, 'max_depth': 6, 'min_samples_split': 11, 'max_features': 0.19}
Blackbox evaluation -> score: 0.9832
Best so far:0.9832

 [Trial8/10]
LLM scores: ['0.98', '0.98', '0.98', '0.98', '0.98']
 -> selected candidates #1: {'n_estimators': 100, 'max_depth': 6, 'min_samples_split': 11, 'max_features': 0.19}
```

```
Blackbox evaluation -> score: 0.9776
Best so far:0.9832

 [Trial9/10]
LLM scores: ['0.98', '0.98', '0.97', '0.98', '0.97']
 -> selected candidates #0: {'n_estimators': 35, 'max_depth': 7, 'min_samples_split': 11, 'max_features': 0.19}
Blackbox evaluation -> score: 0.9832
Best so far:0.9832

 [Trial10/10]
LLM scores: ['0.98', '0.98', '0.98', '0.98', '0.98']
```

```
plt.figure(figsize=(8, 4))
plt.plot(range(1, n_trials+1), history, marker='o', color='steelblue', label='LLAMBO')
plt.axhline(best_score, color='red', linestyle='--', alpha=0.5, label=f'Best={best_score:.4f}')
plt.xlabel("Trial")
plt.ylabel("Best Accuracy (CV)")
plt.title("LLAMBO stype HPO on Iris - Random Forest")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```